INFORMATION AND COMMUNICATION TECHNOLOGIES / ASSIGNMENT 01

Design and Implementation of a Console-Based E-Commerce System using Java

Individual project report
Java OOP: classes, encapsulation, inheritance, collections and UML

STUDENT FULL NAME	Konstantin Permin
STUDENT ID	10624223
COURSE	Information and Communication Technologies (ICT)
SUBMISSION DATE	25/10/2026
TECHNOLOGY	Java / IntelliJ IDEA / draw.io UML

Deliverables: Java source code in packages, editable UML class diagram (draw.io), this documentation report with a full source-code annex, and a runnable console application.

1. Objective and study overview

The objective of this assignment is to design, implement and demonstrate a simple console-based e-commerce system in Java using object-oriented programming. The application supports customers, employees, products and orders. It runs in the IntelliJ IDEA console without external libraries, web frameworks or a database.

The design focuses on classes, constructors, private attributes with controlled getters/setters, inheritance, polymorphism, associations, ArrayList collections, loops, conditional statements and Scanner input, as required by the assignment.

Study objectives

- Model real-world objects as classes with private attributes and appropriate methods.
- Use inheritance: Customer and Employee extend User; Book, Electronics and Clothing extend Product.
- Allow employees to manage users and products, and customers to place and view orders.
- Store users, products and orders in dynamically sized ArrayLists.
- Validate inventory before confirming an order and print a receipt with the total.
- Explain the source code and the UML class diagram during the oral examination.

2. Requirement analysis

Objects and operations were found by reading the assignment text: the **nouns** (user, customer, employee, product, book, electronics, clothing, order) became classes; the **verbs** (add, list, place, view) became methods and menu actions.

Requirement	Solution in the project
User management	Store registers Customer and Employee objects and lists all users.
Product management	Store adds and lists Book, Electronics and Clothing products.
Ordering	A customer selects products and quantities; Store validates the stock and records an Order.
Attributes and methods	Every model class has private fields, a constructor, getters, selected setters and displayInfo()/printReceipt().
Inheritance	Customer/Employee extend User; Book/Electronics/Clothing extend Product.
Encapsulation	All attributes are private; getters/setters give controlled access (see section 3).
Collections	Three ArrayLists in Store hold users, products and orders; Order holds an ArrayList of order lines.
Console interaction	Scanner, while loops and switch statements implement the main, customer and employee menus.
Receipts and history	Order.printReceipt(), Store.listCustomerOrders() and Store.listAllOrders().

Note: one small helper class, ***OrderItem****, was added. It stores one product, its quantity and its unit price, which is needed for an order with several products and keeps the price that was valid when the order was placed.*

3. Object modeling

Class	Private attributes	Key methods / responsibility
User	id, name, email, password, role	getters, setName, setEmail, setPassword, checkPassword, displayInfo
Customer	loyaltyPoints	getLoyaltyPoints, addLoyaltyPoints, displayInfo
Employee	department	getDepartment, setDepartment, displayInfo
Product	id, name, price, stockQuantity	getters, setName, setPrice, setStockQuantity, displayInfo
Book	author	getAuthor, setAuthor, displayInfo
Electronics	warrantyMonths	getWarrantyMonths, setWarrantyMonths, displayInfo
Clothing	size	getSize, setSize, displayInfo
OrderItem	product, quantity, unitPrice	getters, setQuantity, getLineTotal, printLine
Order	orderId, customer, items, totalAmount	getters, calculateTotal, printReceipt
Store	users, products, orders, ID counters	register users, add products, login, placeOrder, list methods
Main	static Scanner, static Store	main, menus, input-validation helpers

All attributes are private. Some values (IDs, roles, order totals) intentionally have no public setter, because uncontrolled changes would break the data. Setters that exist validate their input (for example, negative price or stock is rejected). In this design getters and setters provide *controlled* access instead of making every field freely writable.

Relationships

Relationship	UML type	Multiplicity	In the code
Customer, Employee → User	Inheritance (IS-A)	—	`extends User`
Book, Electronics, Clothing → Product	Inheritance (IS-A)	—	`extends Product`
Order → Customer	Association (navigable)	Order 0..* — 1 Customer	field `customer`
Order ◆ OrderItem	Composition	1 — 1..*	`ArrayList<OrderItem> items` (copies made in the constructor)
OrderItem → Product	Association (navigable)	0..* — 1 Product	field `product`
Store ◇ User / Product / Order	Aggregation	1 — 0..*	three ArrayLists in Store
Main → Store	Association (navigable)	1	static field `store` (Main keeps a Store in a field)
Main ⇢ OrderItem (also Customer, Employee, Product, Order)	Dependency (uses)	—	parameters and local variables, e.g. `new OrderItem(...)` in addToCart
Store ⇢ Book, Electronics, Clothing (also Customer, Employee)	Dependency (creates)	—	`new Book(...)` in addBook; returned from the method

4. UML class diagram

UML (Unified Modeling Language) shows classes, attributes, methods and relationships. Two versions of the diagram are provided:

- **Figure 1** (next page): overview with all classes, their private attributes and the relationships with multiplicities.
- **Appendix B**: complete class diagram with all attributes and methods (A3 landscape, vector graphic).
- The editable source is the draw.io file `diagrams/ECommerce_UML.drawio` with two tabs (overview and full diagram).

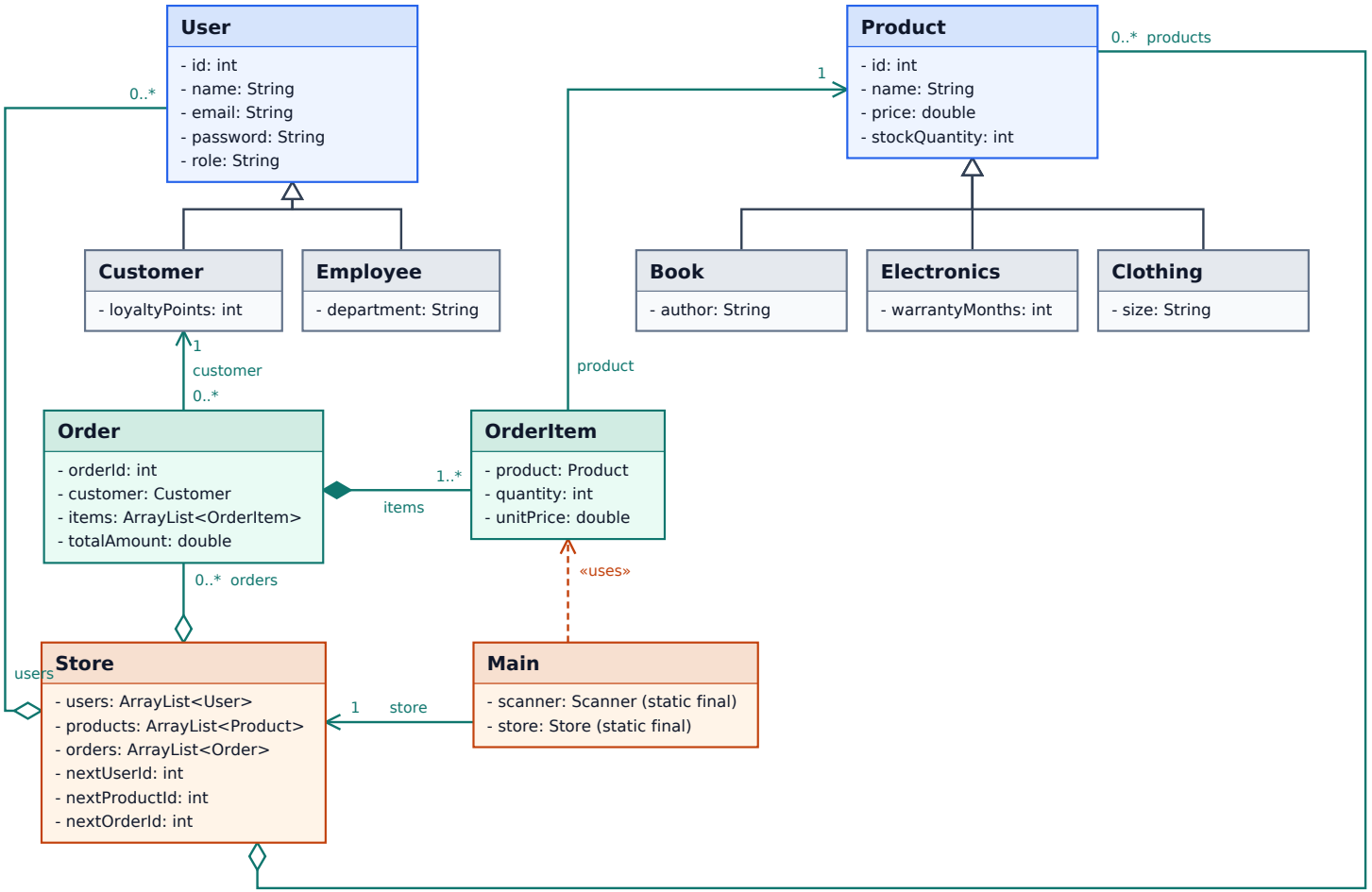
Notation

- **Inheritance (IS-A):** a line with a hollow triangle pointing to the parent class.
- **Association (HAS-A):** a solid line with an open arrow in the direction of navigation; the other class is kept in a *field*, e.g. an Order has a Customer, and Main has a Store. Multiplicities are written at the ends.
- **Aggregation / composition:** a hollow diamond (Store holds users, products and orders) or a filled diamond (an Order owns its order lines).
- **Dependency (USES):** a dashed arrow; a class only uses another one temporarily (parameter, local variable, return type) without keeping it in a field, e.g. Main creates OrderItem objects for the shopping cart.
- **Visibility and types:** "-" private, "+" public; syntax name: type for attributes and name(parameter: type): returnType for methods, for example setPrice(price: double): void.

The class names, attributes and method names in both diagrams are generated from, and checked against, the Java source files in the annex.

ASSIGNMENT 1 / UML CLASS DIAGRAM

Overview: classes, attributes and relationships



"-" private, "+" public | hollow triangle = inheritance (IS-A)
hollow diamond = aggregation | filled diamond = composition
solid line + open arrow = association (kept in a field)
dashed arrow = dependency (parameter, local variable, return type)
Not drawn: Main also uses Customer, Employee, Product, Order;
Store also creates Book, Electronics, Clothing, Customer, Employee.

Figure 1. Parent classes at the top, derived classes below, order associations in the middle and Store/Main at the bottom.

5. Implementation and design decisions

5.1 Project structure

```
ICT_Assignment1_ECommerce/
  src/ict/ecommerce/
    Main.java
    Store.java
    model/
      User.java, Customer.java, Employee.java
      Product.java, Book.java, Electronics.java, Clothing.java
      Order.java, OrderItem.java
  diagrams/ECommerce_UML.drawio (+ PNG/SVG)
  docs/Assignment1_Report.pdf
```

The *model* package contains the entities, *Store* holds and manages the data, and *Main* handles console interaction. This separation avoids putting the whole program in one main method.

5.2 Inheritance, encapsulation and polymorphism

- **Inheritance:** subclass constructors call `super(...)` to initialize the parent fields.
- **Encapsulation:** all fields are private. For example, setStockQuantity() prevents negative stock, and the password is checked with checkPassword() instead of being returned by a getter.
- **Overriding:** each subclass overrides displayInfo() and calls `super.displayInfo()` before printing its own data.
- **Polymorphism:** an ArrayList<Product> holds Book, Electronics and Clothing objects; product.displayInfo() calls the correct override at run time.

5.3 Console interface and roles

The main menu allows customer registration, login and product browsing. After login a customer can list products, place an order, view their own orders and view their profile. An employee can add books, electronics and clothing, list users and products, register customers and employees, and view all orders. The helper methods readInt, readPositiveInt, readNonNegativeInt, readPositiveDouble and readText reject invalid input in loops. Every menu is a while loop with a switch statement.

5.4 Order workflow

Step	Action
1	The customer chooses a product ID and a positive quantity.
2	Main puts an OrderItem into the cart, or increases the quantity of an existing line.
3	The customer sees a preview and confirms or cancels.
4	Store checks the available stock of every line **before** changing any stock.
5	A new Order is created, stock is decreased and the order is added to the ArrayList.
6	Order.printReceipt() prints customer, products, quantities and the total.

OrderItem keeps a snapshot of the unit price, so a saved receipt keeps its original prices even if the catalog changes later.

6. Testing and results

The project was compiled with JDK 21 and run with scripted console input. The following scenarios were checked:

Test	Description	Result
T01	Guest product listing	PASS
T02	Incorrect login credentials are rejected	PASS
T03	Employee adds all three product types and both user types	PASS

Test	Description	Result
T04	Order: 2 books + 1 headphones gives 29,000 KZT and reduces stock	PASS
T05	Quantity greater than stock is rejected	PASS
T06	Cancelling an order leaves stock unchanged	PASS
T07	Duplicate cart entries are merged; excess quantity is rejected	PASS
T08	Non-numeric menu input is handled	PASS
T09	Duplicate e-mail is rejected	PASS

Verified receipt (T04)

```
RECEIPT | Order #1
Customer: Demo Customer
  Java Basics x 2 @ 7000.0 KZT = 14000.0 KZT
  Headphones x 1 @ 15000.0 KZT = 15000.0 KZT
TOTAL: 29000.0 KZT
```

After this order the stock of Java Basics decreases from 6 to 4 and the stock of Headphones from 4 to 3.

7. Limitations and possible improvements

- **Storage:** data is kept in memory only and is lost when the program closes (the assignment scope is ArrayList-based).
- **Security:** demo passwords are simple and stored as plain text. A real system needs password hashing, role control and stronger validation.
- **Currency:** double is used for simplicity; BigDecimal is preferable for real money calculations.
- **Concurrency:** the program is a single-user console simulation.
- **Out of scope:** database, payment processing, web interface and third-party libraries.

8. Conclusion

The Java console e-commerce system fulfils the functional and object-oriented requirements of Assignment 1. It demonstrates classes and objects, inheritance, encapsulation, polymorphism, associations, constructors, ArrayList collections, Scanner-based input, control flow, order creation and receipt generation. The program was compiled and tested, and the UML diagrams match the implemented classes. The annex contains every Java source file.

9. Running the application

Open the project folder in IntelliJ IDEA, choose JDK 17 or later and run the configuration **ECommerce - Main** (main class `ict.ecommerce.Main`). Demo logins: employee `employee@demo.com / 1234`, customer `customer@demo.com / 1234`.

Appendix A. Complete Java source code

All eleven Java source files of the project are reproduced below (the assignment allows the source code as an annex instead of a Git address).

A.1 src/ict/ecommerce/model/User.java

```
 1  package ict.ecommerce.model;
 2
 3  // Parent class for both customers and employees.
 4  public class User {
 5      private int id;
 6      private String name;
 7      private String email;
 8      private String password;
 9      private String role;
10
11      public User(int id, String name, String email, String password, String role) {
12          this.id = id;
13          this.name = name;
14          this.email = email;
15          this.password = password;
16          this.role = role;
17      }
18
19      public int getId() {
20          return id;
21      }
22
23      public String getName() {
24          return name;
25      }
26
27      public void setName(String name) {
28          this.name = name;
29      }
30
31      public String getEmail() {
32          return email;
33      }
34
35      public void setEmail(String email) {
36          this.email = email;
37      }
38
39      public String getRole() {
40          return role;
41      }
42
43      // Password is checked, not printed or returned by a getter.
44      public boolean checkPassword(String enteredPassword) {
45          return password.equals(enteredPassword);
46      }
47
48      public void setPassword(String newPassword) {
49          this.password = newPassword;
50      }
51
52      public void displayInfo() {
53          System.out.println("User #" + id + " | " + name + " | " + email + " | " + role);
54      }
55  }
```

A.2 src/ict/ecommerce/model/Customer.java

```
 1  package ict.ecommerce.model;
 2
 3  // A customer IS-A user.
 4  public class Customer extends User {
 5      private int loyaltyPoints;
 6
 7      public Customer(int id, String name, String email, String password) {
 8          super(id, name, email, password, "Customer");
 9          this.loyaltyPoints = 0;
10      }
11
12      public int getLoyaltyPoints() {
13          return loyaltyPoints;
14      }
15
16      public void addLoyaltyPoints(int points) {
17          if (points > 0) {
18              loyaltyPoints += points;
19          }
20      }
21
22      @Override
23      public void displayInfo() {
24          super.displayInfo();
25          System.out.println("  Loyalty points: " + loyaltyPoints);
26      }
27  }
```

A.3 src/ict/ecommerce/model/Employee.java

```
 1  package ict.ecommerce.model;
 2
 3  // An employee IS-A user.
 4  public class Employee extends User {
 5      private String department;
 6
 7      public Employee(int id, String name, String email, String password, String department) {
 8          super(id, name, email, password, "Employee");
 9          this.department = department;
10      }
11
12      public String getDepartment() {
13          return department;
14      }
15
16      public void setDepartment(String department) {
17          this.department = department;
18      }
19
20      @Override
21      public void displayInfo() {
22          super.displayInfo();
23          System.out.println("  Department: " + department);
24      }
25  }
```

A.4 src/ict/ecommerce/model/Product.java

```
 1  package ict.ecommerce.model;
 2
 3  // Parent class for books, electronics and clothing.
 4  public class Product {
 5      private int id;
 6      private String name;
 7      private double price;
 8      private int stockQuantity;
 9
10      public Product(int id, String name, double price, int stockQuantity) {
11          this.id = id;
12          this.name = name;
13          if (price >= 0) {
14              this.price = price;
15          }
16          if (stockQuantity >= 0) {
17              this.stockQuantity = stockQuantity;
18          }
19      }
20
21      public int getId() {
22          return id;
23      }
24
25      public String getName() {
26          return name;
27      }
28
29      public void setName(String name) {
30          this.name = name;
31      }
32
33      public double getPrice() {
34          return price;
35      }
36
37      // Negative prices are not allowed.
38      public void setPrice(double price) {
39          if (price >= 0) {
40              this.price = price;
41          }
42      }
43
44      public int getStockQuantity() {
45          return stockQuantity;
46      }
47
48      // Negative stock is not allowed.
49      public void setStockQuantity(int stockQuantity) {
50          if (stockQuantity >= 0) {
51              this.stockQuantity = stockQuantity;
52          }
53      }
54
55      public void displayInfo() {
56          System.out.println("Product #" + id + " | " + name + " | " + price
57                  + " KZT | Stock: " + stockQuantity);
58      }
59  }
```

A.5 src/ict/ecommerce/model/Book.java

```
 1  package ict.ecommerce.model;
 2
 3  // A book IS-A product.
 4  public class Book extends Product {
 5      private String author;
 6
 7      public Book(int id, String name, double price, int stockQuantity, String author) {
 8          super(id, name, price, stockQuantity);
 9          this.author = author;
10      }
11
12      public String getAuthor() {
13          return author;
14      }
15
16      public void setAuthor(String author) {
17          this.author = author;
18      }
19
20      @Override
21      public void displayInfo() {
22          super.displayInfo();
23          System.out.println("  Type: Book | Author: " + author);
24      }
25  }
```

A.6 src/ict/ecommerce/model/Electronics.java

```
 1  package ict.ecommerce.model;
 2
 3  // Electronics IS-A product.
 4  public class Electronics extends Product {
 5      private int warrantyMonths;
 6
 7      public Electronics(int id, String name, double price, int stockQuantity, int warrantyMonths) {
 8          super(id, name, price, stockQuantity);
 9          this.warrantyMonths = warrantyMonths;
10      }
11
12      public int getWarrantyMonths() {
13          return warrantyMonths;
14      }
15
16      public void setWarrantyMonths(int warrantyMonths) {
17          if (warrantyMonths >= 0) {
18              this.warrantyMonths = warrantyMonths;
19          }
20      }
21
22      @Override
23      public void displayInfo() {
24          super.displayInfo();
25          System.out.println("  Type: Electronics | Warranty: " + warrantyMonths + " months");
26      }
27  }
```

A.7 src/ict/ecommerce/model/Clothing.java

```
 1  package ict.ecommerce.model;
 2
 3  // Clothing IS-A product.
 4  public class Clothing extends Product {
 5      private String size;
 6
 7      public Clothing(int id, String name, double price, int stockQuantity, String size) {
 8          super(id, name, price, stockQuantity);
 9          this.size = size;
10      }
11
12      public String getSize() {
13          return size;
14      }
15
16      public void setSize(String size) {
17          this.size = size;
18      }
19
20      @Override
21      public void displayInfo() {
22          super.displayInfo();
23          System.out.println("  Type: Clothing | Size: " + size);
24      }
25  }
```

A.8 src/ict/ecommerce/model/OrderItem.java

```
 1  package ict.ecommerce.model;
 2
 3  // One line in an order: which product, how many, and at what price.
 4  public class OrderItem {
 5      private Product product;
 6      private int quantity;
 7      private double unitPrice;
 8
 9      public OrderItem(Product product, int quantity) {
10          this.product = product;
11          this.quantity = quantity;
12          this.unitPrice = product.getPrice();
13      }
14
15      public Product getProduct() {
16          return product;
17      }
18
19      public int getQuantity() {
20          return quantity;
21      }
22
23      public void setQuantity(int quantity) {
24          if (quantity > 0) {
25              this.quantity = quantity;
26          }
27      }
28
29      public double getUnitPrice() {
30          return unitPrice;
31      }
32
33      public double getLineTotal() {
34          return unitPrice * quantity;
35      }
36
37      public void printLine() {
38          System.out.println("  " + product.getName() + " x " + quantity + " @ "
39                  + unitPrice + " KZT = " + getLineTotal() + " KZT");
40      }
41  }
```

A.9 src/ict/ecommerce/model/Order.java

```
 1  package ict.ecommerce.model;
 2
 3  import java.util.ArrayList;
 4
 5  // An order HAS-A customer and HAS multiple order items.
 6  public class Order {
 7      private int orderId;
 8      private Customer customer;
 9      private ArrayList<OrderItem> items;
10      private double totalAmount;
11
12      public Order(int orderId, Customer customer, ArrayList<OrderItem> cart) {
13          this.orderId = orderId;
14          this.customer = customer;
15          this.items = new ArrayList<>();
16
17          // Copy order lines so later cart changes do not change an existing order.
18          double total = 0;
19          for (OrderItem item : cart) {
20              OrderItem copy = new OrderItem(item.getProduct(), item.getQuantity());
21              items.add(copy);
22              total += copy.getLineTotal();
23          }
24          this.totalAmount = total;
25      }
26
27      public int getOrderId() {
28          return orderId;
29      }
30
31      public Customer getCustomer() {
32          return customer;
33      }
34
35      public double getTotalAmount() {
36          return totalAmount;
37      }
38
39      public double calculateTotal() {
40          double total = 0;
41          for (OrderItem item : items) {
42              total += item.getLineTotal();
43          }
44          return total;
45      }
46
47      public void printReceipt() {
48          System.out.println("----------------------------------------");
49          System.out.println("RECEIPT | Order #" + orderId);
50          System.out.println("Customer: " + customer.getName());
51          for (OrderItem item : items) {
52              item.printLine();
53          }
54          System.out.println("TOTAL: " + totalAmount + " KZT");
55          System.out.println("----------------------------------------");
56      }
57  }
```

A.10 src/ict/ecommerce/Store.java

```
package ict.ecommerce;

import ict.ecommerce.model.*;
import java.util.ArrayList;

// Stores users, products and orders in memory using ArrayList.
public class Store {
    private ArrayList<User> users;
    private ArrayList<Product> products;
    private ArrayList<Order> orders;
    private int nextUserId;
    private int nextProductId;
    private int nextOrderId;

    public Store() {
        users = new ArrayList<>();
        products = new ArrayList<>();
        orders = new ArrayList<>();
        nextUserId = 1;
        nextProductId = 1;
        nextOrderId = 1;

        // Demo records make the application easy to demonstrate in a lab.
        users.add(new Customer(nextUserId, "Demo Customer", "customer@demo.com", "1234"));
        nextUserId++;
        users.add(new Employee(nextUserId, "Demo Employee", "employee@demo.com", "1234", "Sales"));
        nextUserId++;
        products.add(new Book(nextProductId, "Java Basics", 7000.0, 6, "A. Teacher"));
        nextProductId++;
        products.add(new Electronics(nextProductId, "Headphones", 15000.0, 4, 12));
        nextProductId++;
        products.add(new Clothing(nextProductId, "T-Shirt", 5500.0, 8, "M"));
        nextProductId++;
    }

    public boolean emailExists(String email) {
        for (User user : users) {
            if (user.getEmail().equalsIgnoreCase(email)) {
                return true;
            }
        }
        return false;
    }

    public Customer registerCustomer(String name, String email, String password) {
        if (emailExists(email)) {
            return null;
        }
        Customer customer = new Customer(nextUserId, name, email, password);
        nextUserId++;
        users.add(customer);
        return customer;
    }

    public Employee registerEmployee(String name, String email, String password, String department) {
        if (emailExists(email)) {
            return null;
        }
        Employee employee = new Employee(nextUserId, name, email, password, department);
        nextUserId++;
        users.add(employee);
        return employee;
    }

    public User login(String email, String password) {
        for (User user : users) {
            if (user.getEmail().equalsIgnoreCase(email) && user.checkPassword(password)) {
                return user;
            }
        }
        return null;
    }

    public void listUsers() {
        System.out.println("\n=== ALL USERS ===");
        for (User user : users) {
            user.displayInfo();
        }
```

```
    }

    public Book addBook(String name, double price, int stock, String author) {
        Book book = new Book(nextProductId, name, price, stock, author);
        nextProductId++;
        products.add(book);
        return book;
    }

    public Electronics addElectronics(String name, double price, int stock, int warranty) {
        Electronics electronics = new Electronics(nextProductId, name, price, stock, warranty);
        nextProductId++;
        products.add(electronics);
        return electronics;
    }

    public Clothing addClothing(String name, double price, int stock, String size) {
        Clothing clothing = new Clothing(nextProductId, name, price, stock, size);
        nextProductId++;
        products.add(clothing);
        return clothing;
    }

    public Product findProductById(int id) {
        for (Product product : products) {
            if (product.getId() == id) {
                return product;
            }
        }
        return null;
    }

    public void listProducts() {
        System.out.println("\n=== AVAILABLE PRODUCTS ===");
        if (products.isEmpty()) {
            System.out.println("No products available.");
        }
        for (Product product : products) {
            product.displayInfo();
        }
    }

    // First check stock for EVERY line. Only then modify stock and create the order.
    public Order placeOrder(Customer customer, ArrayList<OrderItem> cart) {
        if (cart.isEmpty()) {
            return null;
        }
        for (OrderItem item : cart) {
            if (item.getQuantity() <= 0
                    || item.getQuantity() > item.getProduct().getStockQuantity()) {
                return null;
            }
        }

        // No validation failed: the order can now be completed.
        Order order = new Order(nextOrderId, customer, cart);
        nextOrderId++;
        for (OrderItem item : cart) {
            Product product = item.getProduct();
            product.setStockQuantity(product.getStockQuantity() - item.getQuantity());
        }
        orders.add(order);
        customer.addLoyaltyPoints(1);
        return order;
    }

    public void listAllOrders() {
        System.out.println("\n=== ALL ORDERS ===");
        if (orders.isEmpty()) {
            System.out.println("No orders have been placed yet.");
        }
        for (Order order : orders) {
            order.printReceipt();
        }
    }

    public void listCustomerOrders(Customer customer) {
        System.out.println("\n=== MY ORDERS ===");
        boolean found = false;
        for (Order order : orders) {
```

```
159              if (order.getCustomer().getId() == customer.getId()) {
160                  order.printReceipt();
161                  found = true;
162              }
163          }
164          if (!found) {
165              System.out.println("You do not have any orders yet.");
166          }
167      }
168  }
```

A.11 src/ict/ecommerce/Main.java

```
 1  package ict.ecommerce;
 2
 3  import ict.ecommerce.model.*;
 4  import java.util.ArrayList;
 5  import java.util.Locale;
 6  import java.util.Scanner;
 7
 8  // Console interface: input, menus and messages are handled here.
 9  public class Main {
10      private static final Scanner scanner = new Scanner(System.in);
11      private static final Store store = new Store();
12
13      public static void main(String[] args) {
14          // Use a period as the decimal separator for prices (e.g. 1999.50).
15          scanner.useLocale(Locale.US);
16          System.out.println("Welcome to the ICT Console E-Commerce System!");
17          System.out.println("DEMO customer: customer@demo.com / 1234");
18          System.out.println("DEMO employee: employee@demo.com / 1234");
19
20          boolean running = true;
21          while (running) {
22              System.out.println("\n========== MAIN MENU ==========");
23              System.out.println("1. Register as customer");
24              System.out.println("2. Log in");
25              System.out.println("3. Browse products");
26              System.out.println("0. Exit");
27              int choice = readInt("Choose: ");
28
29              switch (choice) {
30                  case 1:
31                      registerCustomer();
32                      break;
33                  case 2:
34                      login();
35                      break;
36                  case 3:
37                      store.listProducts();
38                      break;
39                  case 0:
40                      running = false;
41                      break;
42                  default:
43                      System.out.println("Invalid menu option.");
44              }
45          }
46          scanner.close();
47          System.out.println("Goodbye!");
48      }
49
50      private static void registerCustomer() {
51          System.out.println("\n--- REGISTER CUSTOMER ---");
52          String name = readText("Full name: ");
53          String email = readText("Email: ");
54          if (store.emailExists(email)) {
55              System.out.println("This email is already registered.");
56              return;
57          }
58          String password = readText("Password: ");
59          Customer customer = store.registerCustomer(name, email, password);
60          System.out.println("Customer added! ID: " + customer.getId());
61      }
62
63      private static void registerEmployee() {
64          System.out.println("\n--- ADD EMPLOYEE ---");
65          String name = readText("Full name: ");
66          String email = readText("Email: ");
67          if (store.emailExists(email)) {
68              System.out.println("This email is already registered.");
69              return;
70          }
71          String password = readText("Password: ");
72          String department = readText("Department: ");
73          Employee employee = store.registerEmployee(name, email, password, department);
74          System.out.println("Employee added! ID: " + employee.getId());
75      }
76
77      private static void login() {
78          System.out.println("\n--- LOGIN ---");
```

```
        String email = readText("Email: ");
        String password = readText("Password: ");
        User user = store.login(email, password);
        if (user == null) {
            System.out.println("Wrong email or password.");
        } else if (user instanceof Customer) {
            customerMenu((Customer) user);
        } else if (user instanceof Employee) {
            employeeMenu((Employee) user);
        }
    }

    private static void customerMenu(Customer customer) {
        boolean logout = false;
        while (!logout) {
            System.out.println("\n===== CUSTOMER: " + customer.getName() + " =====");
            System.out.println("1. List products");
            System.out.println("2. Place order");
            System.out.println("3. View my orders");
            System.out.println("4. View my profile");
            System.out.println("0. Log out");
            int choice = readInt("Choose: ");

            switch (choice) {
                case 1:
                    store.listProducts();
                    break;
                case 2:
                    placeOrder(customer);
                    break;
                case 3:
                    store.listCustomerOrders(customer);
                    break;
                case 4:
                    customer.displayInfo();
                    break;
                case 0:
                    logout = true;
                    break;
                default:
                    System.out.println("Invalid menu option.");
            }
        }
    }

    private static void employeeMenu(Employee employee) {
        boolean logout = false;
        while (!logout) {
            System.out.println("\n===== EMPLOYEE: " + employee.getName() + " =====");
            System.out.println("1. List products");
            System.out.println("2. Add book");
            System.out.println("3. Add electronics");
            System.out.println("4. Add clothing");
            System.out.println("5. List all users");
            System.out.println("6. Add customer");
            System.out.println("7. Add employee");
            System.out.println("8. View all orders");
            System.out.println("0. Log out");
            int choice = readInt("Choose: ");

            switch (choice) {
                case 1:
                    store.listProducts();
                    break;
                case 2:
                    addBook();
                    break;
                case 3:
                    addElectronics();
                    break;
                case 4:
                    addClothing();
                    break;
                case 5:
                    store.listUsers();
                    break;
                case 6:
                    registerCustomer();
                    break;
                case 7:
```

```
                    registerEmployee();
                    break;
                case 8:
                    store.listAllOrders();
                    break;
                case 0:
                    logout = true;
                    break;
                default:
                    System.out.println("Invalid menu option.");
            }
        }
    }

    private static void addBook() {
        System.out.println("\n--- ADD BOOK ---");
        String name = readText("Name: ");
        double price = readPositiveDouble("Price (KZT): ");
        int stock = readNonNegativeInt("Stock quantity: ");
        String author = readText("Author: ");
        Book book = store.addBook(name, price, stock, author);
        System.out.println("Book added with ID " + book.getId());
    }

    private static void addElectronics() {
        System.out.println("\n--- ADD ELECTRONICS ---");
        String name = readText("Name: ");
        double price = readPositiveDouble("Price (KZT): ");
        int stock = readNonNegativeInt("Stock quantity: ");
        int warranty = readNonNegativeInt("Warranty in months: ");
        Electronics electronics = store.addElectronics(name, price, stock, warranty);
        System.out.println("Electronics added with ID " + electronics.getId());
    }

    private static void addClothing() {
        System.out.println("\n--- ADD CLOTHING ---");
        String name = readText("Name: ");
        double price = readPositiveDouble("Price (KZT): ");
        int stock = readNonNegativeInt("Stock quantity: ");
        String size = readText("Size (S/M/L/XL): ");
        Clothing clothing = store.addClothing(name, price, stock, size);
        System.out.println("Clothing added with ID " + clothing.getId());
    }

    private static void placeOrder(Customer customer) {
        ArrayList<OrderItem> cart = new ArrayList<>();
        System.out.println("\n--- NEW ORDER (0 = finish selecting) ---");
        boolean selecting = true;

        while (selecting) {
            store.listProducts();
            int id = readInt("Product ID (0 to finish): ");
            if (id == 0) {
                selecting = false;
            } else {
                Product product = store.findProductById(id);
                if (product == null) {
                    System.out.println("Product not found.");
                } else if (product.getStockQuantity() == 0) {
                    System.out.println("This product is out of stock.");
                } else {
                    int quantity = readPositiveInt("Quantity: ");
                    addToCart(cart, product, quantity);
                }
            }
        }

        if (cart.isEmpty()) {
            System.out.println("Cart is empty. Order cancelled.");
            return;
        }

        double previewTotal = 0;
        System.out.println("\n--- ORDER PREVIEW ---");
        for (OrderItem item : cart) {
            item.printLine();
            previewTotal += item.getLineTotal();
        }
        System.out.println("TOTAL: " + previewTotal + " KZT");
        int confirmation = readInt("Confirm order? 1 = yes, 0 = no: ");
```

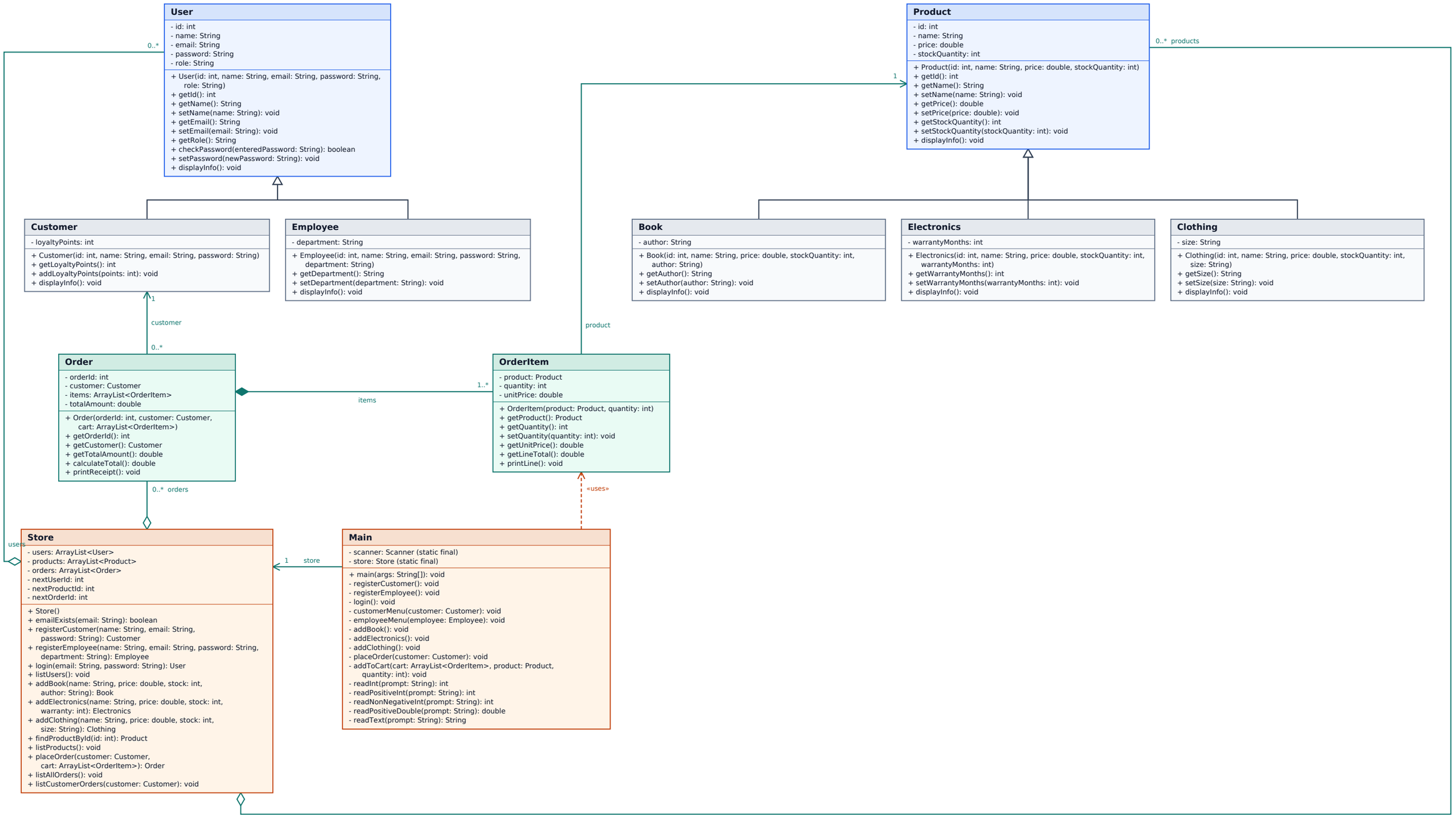
```
239         if (confirmation != 1) {
240             System.out.println("Order cancelled. Stock was not changed.");
241             return;
242         }
243 
244         Order order = store.placeOrder(customer, cart);
245         if (order == null) {
246             System.out.println("Order failed: check stock quantities.");
247         } else {
248             System.out.println("Order placed successfully!");
249             order.printReceipt();
250         }
251     }
252 
253     private static void addToCart(ArrayList<OrderItem> cart, Product product, int quantity) {
254         // If the product is already in the cart, increase its quantity.
255         for (OrderItem item : cart) {
256             if (item.getProduct().getId() == product.getId()) {
257                 if (quantity > product.getStockQuantity() - item.getQuantity()) {
258                     System.out.println("Not enough stock. Available: " + product.getStockQuantity());
259                 } else {
260                     int totalQuantity = item.getQuantity() + quantity;
261                     item.setQuantity(totalQuantity);
262                     System.out.println("Updated quantity in cart: " + totalQuantity);
263                 }
264                 return;
265             }
266         }
267 
268         if (quantity > product.getStockQuantity()) {
269             System.out.println("Not enough stock. Available: " + product.getStockQuantity());
270         } else {
271             cart.add(new OrderItem(product, quantity));
272             System.out.println("Added to cart.");
273         }
274     }
275 
276     // All numeric input uses Scanner; invalid tokens are safely skipped.
277     private static int readInt(String prompt) {
278         while (true) {
279             System.out.print(prompt);
280             if (scanner.hasNextInt()) {
281                 int value = scanner.nextInt();
282                 scanner.nextLine();
283                 return value;
284             }
285             System.out.println("Enter a valid whole number.");
286             scanner.nextLine();
287         }
288     }
289 
290     private static int readPositiveInt(String prompt) {
291         int value = readInt(prompt);
292         while (value <= 0) {
293             System.out.println("Number must be greater than zero.");
294             value = readInt(prompt);
295         }
296         return value;
297     }
298 
299     private static int readNonNegativeInt(String prompt) {
300         int value = readInt(prompt);
301         while (value < 0) {
302             System.out.println("Number cannot be negative.");
303             value = readInt(prompt);
304         }
305         return value;
306     }
307 
308     private static double readPositiveDouble(String prompt) {
309         while (true) {
310             System.out.print(prompt);
311             if (scanner.hasNextDouble()) {
312                 double value = scanner.nextDouble();
313                 scanner.nextLine();
314                 if (value > 0 && !Double.isInfinite(value)) {
315                     return value;
316                 }
317             } else {
318                 scanner.nextLine();
```

```
319             }
320             System.out.println("Enter a positive price, using a dot (e.g. 1500.50).");
321         }
322     }
323
324     private static String readText(String prompt) {
325         String value = "";
326         while (value.trim().isEmpty()) {
327             System.out.print(prompt);
328             value = scanner.nextLine().trim();
329             if (value.isEmpty()) {
330                 System.out.println("Text cannot be empty.");
331             }
332         }
333         return value;
334     }
335 }
```

Appendix B. Complete UML class diagram (all classes, attributes and methods)

APPENDIX B / COMPLETE UML CLASS DIAGRAM

All classes with attributes, methods (typed parameters), inheritance, associations and dependencies



Notation: "-" private, "+" public | hollow triangle = inheritance (IS-A) | hollow diamond = aggregation | filled diamond = composition
solid line with open arrow = association (class keeps the other class in a field) | dashed arrow = dependency (uses it as a parameter, local variable or return type)
Dependencies not drawn: Main also uses Customer, Employee, Product and Order; Store also creates Book, Electronics, Clothing, Customer and Employee.